Entmoot: Social Group Communication for Autonomous Agents over Pilot Protocol

Jerry Fanelli
jerry.fanelli@gmail.com

June 11, 2026

Abstract

Recent empirical analysis of the Pilot Protocol agent network shows that autonomous agents can self-organize into a small-world social graph on pairwise trust alone [5]. Pilot supplies identity, trust, discovery, NAT traversal, and encrypted pairwise transport, but it does not provide durable group communication: no signed shared roster, no topic log, no public discovery mechanism, no policy layer, and no searchable history for intermittent clients. We present *Entmoot*, a social group communication layer for autonomous agents over Pilot. Entmoot represents a group, or *moot*, as signed membership state plus signed messages propagated by gossip and repaired by anti-entropy reconciliation. A reference Go implementation provides private and public moots, targeted and open invites, founder-managed policies, an Entmoot Service Provider (ESP) surface for web and mobile clients, lexical search with message-context jumps, and Agent Skills plus Codex/Claude plugin packaging for agent-facing use. The system is deployed as working software. This paper reports its design, implementation, and a 120-hour live multi-agent evaluation across a VPS, two container runtimes, and a residential Raspberry Pi. The run collected 245 health snapshots, ended with all four nodes reporting healthy doctor and peer checks, and exposed intermittent residential-edge monitor reachability as the main operational limitation.

1 Introduction

Autonomous agents already form social structure. In an empirical study of the Pilot Protocol network, agents registered, handshook, exchanged trust, and formed a topology with heavy-tailed degree distribution, 47× higher clustering than random, and a giant component spanning two-thirds of the population [5]. That result is striking because Pilot offers pairwise primitives rather than an application-level social substrate: each agent can discover another node, negotiate trust, and open an encrypted 1-to-1 tunnel. What persists across more than two peers is left to applications.

The missing primitive is a durable group conversation. A pairwise overlay can connect agents, but it does not by itself answer group questions: who belongs to a room, who may invite new members, which messages were sent on a topic, how intermittent clients catch up, how a public room is discovered, or how an agent searches shared history after the fact. An agent that wants to address a dozen peers can maintain individual relationships with all of them,

or it can delegate group state to a centralized service. The first approach scales poorly as a social practice; the second undermines the decentralized trust model that made the Pilot graph interesting in the first place.

Entmoot adds that group layer without replacing Pilot. A group, or *moot*, consists of signed roster state, signed messages, optional invites, policy metadata, and a gossip/reconciliation protocol that moves data over ordinary Pilot streams. Members can participate directly, while an Entmoot Service Provider (ESP) can project the same group state to web and mobile clients without becoming the source of group truth. Public moot descriptors make opt-in communities discoverable, but public visibility is separate from message history, live replies, and membership consent.

The current implementation is social-first. It supports private and public moots, default and founder-customized policies, searchable history, message-context jumps from search results back into chat, member profiles and display names, owner-controlled live-agent replies, and an Agent Skills-compatible packaging surface for Codex, Claude, and OpenClaw-style agents. Fleet and task coordination exist only as opt-in extensions and are disabled by default. This framing matters: Entmoot is not primarily a job orchestrator. It is a shared social memory and conversation layer for agents.

Our contributions are as follows.

1. A signed moot model for group communication over Pilot, including member identities, rosters, targeted invites, open invites, signed messages, topic routing, and founder/admin authority.
2. A dissemination and repair design that combines Pilot streams, gossip fanout, replay protection, rate limiting, Merkle roots, and anti-entropy reconciliation to make group history durable across intermittently connected agents.
3. An ESP architecture that exposes group state to web and mobile clients while preserving the distinction between signed group state and ESP-local convenience state such as device auth, cursors, profiles, and display names.
4. Public discovery and governance mechanisms: public moot descriptors, ESP directory indexing, consent-first joins, default policies, and founder-managed policy updates.
5. Agent-facing packaging through an Agent Skills document and Codex/Claude plugin generation, so autonomous coding agents can discover, install, diagnose, and use Entmoot as a chat substrate.
6. A five-day empirical evaluation that measures multi-runtime monitoring, controlled-topic delivery, transcript export, and recovery behavior across heterogeneous agent runtimes.

The rest of the paper is organized as follows. Section 2 positions Entmoot against Pilot, pub/sub, social, gossip, and log-repair systems. Section 3 defines the system model and trust boundaries. Sections 4 and 5 describe the protocol and implementation. Sections 6 and 7 separate the evaluation method from the results. Sections 8, 9, and 10 discuss the implications, limits, and artifact record before the future-work and conclusion sections.

2 Background and Related Work

2.1 Pilot Protocol

Pilot is an overlay network stack for autonomous agents [30]. Each participating node generates an Ed25519 identity [3, 13] at registration and receives a 48-bit virtual address of the form

`N:NNNN.HHHH.LLLL`: a 16-bit network ID and a 32-bit node ID within that network. Network 0 is the global backbone; all agents are members of it by default. Pilot reserves additional network IDs for purpose-specific subgraphs; the *Pilot Networks* feature that populates them is in early-access release as of this writing. A Pilot Network is a group-scoped *connectivity* primitive: membership grants pairwise authorization ("address resolution, connection acceptance, and datagram delivery" [30]) between members without requiring $O(N^2)$ bilateral handshakes. It is not a messaging layer: Pilot Networks do not define a broadcast primitive, an ordered log, retention, or completeness proofs — datagrams between network members are still point-to-point and stateless. Entmoot and Pilot Networks are therefore complementary rather than overlapping: Pilot Networks can eventually replace Entmoot's signed roster as the membership source of truth (§11), while Entmoot contributes the broadcast semantics — fanout, Merkle-verified completeness, and anti-entropy reconciliation — that a Network membership primitive alone does not provide.

Communication is trust-gated. Two nodes must complete a cryptographic handshake (port 444) mediated by the registry before they can exchange traffic on any other port [5]. Once trust is established, nodes may open a reliable byte stream to each other on any port. Under the hood, Pilot runs a TCP-equivalent reliable-delivery protocol with X25519 key exchange [15] and AES-256-GCM authenticated encryption [10] per tunnel, the same primitives that underlie WireGuard [8]. The bytes that carry a tunnel move through a pluggable transport layer: UDP is the default byte-mover and is what UDP-capable peers use end-to-end, but the fork we build against also ships a TCP transport that a public-IP daemon can expose alongside UDP so peers on UDP-hostile networks (restrictive corporate firewalls, carrier-grade NAT, UDP-blocking ISPs) can still establish tunnels. The transport choice is invisible above the tunnel layer: the same wire frames travel over whichever transport the dialing peer's direct-UDP retries settle on. The registry does not see tunnel payloads; it only mediates address allocation, hostname resolution, the handshake relay, and, when present, the per-peer list of transport endpoints.

Crucially, the registry hides the network endpoint of any node not flagged as "public." A private node's address can only be resolved to a reachable endpoint by another node with an existing trust edge. A new participant cannot therefore bootstrap into an otherwise-private group without either an out-of-band introduction or at least one public introducer.

We build against the `jerryfane/pilotprotocol` fork [30], which layers a pluggable transport abstraction and a cluster of reliability fixes on top of upstream v1.7.2. The fork now includes NAT-drift-resilient endpoint refresh (a STUN-style guard against carrier NAT remapping [26]), TURN client routing for peers behind symmetric NAT [27], strict `-outbound-turn-only` privacy mode, a small rendezvous service for TURN-endpoint discovery, periodic peer re-lookup, TURN permission refresh/self-heal, explicit virtual-stream lifecycle tracing, and a self-dial guard at the transport layer (§4.6). Entmoot treats these as lower-layer capabilities: rendezvous discovers endpoints, Pilot chooses and executes routes, and Entmoot consumes only reliable Pilot streams and the daemon IPC surface. The companion evolution paper [11] records the incident-level narrative of how these were surfaced in live deployment and shaped the fanout, multiplexing, anti-entropy, and recovery policies described in §4.

2.2 Autonomous agent networks

Classical multi-agent systems literature [34, 28, 9] models agents, strategies, and coordination

protocols, but it usually assumes that the interaction topology and communication substrate are already given. Calin [5] instead observes a deployed agent population on Pilot, where the topology itself emerges from trust and discovery decisions: agents form heavy-tailed degree distributions consistent with preferential attachment [1], clustering coefficient $\sim 47\times$ above random, and small-world properties [32]. Functional clusters such as data/analytics, wellness, career, and engineering emerge without a global planner. That study also identifies a limit: all observed agents remain on Pilot network 0, so community structure beyond pairwise trust is not yet a first-class protocol object. Entmoot instantiates that missing group substrate.

2.3 Group messaging, pub/sub, and social protocols

Existing messaging systems supply useful contrasts. MQTT defines hierarchical topics and wildcard filters [25]; NATS uses subject-based publish/subscribe for one-to-many distribution [24]; Matrix specifies federated rooms, event graphs, and client/server access patterns [21]; and ActivityPub standardizes federated social inbox/outbox delivery [19]. Entmoot borrows the idea that messages need addressable topics and client-facing projections, but differs in where authority sits. It is not a brokered pub/sub system like MQTT or NATS, and its ESP layer is not the canonical home server of a Matrix-style room. Signed rosters, signed messages, public descriptors, and reconciliation remain verifiable outside the service that presents them to a web or mobile client. Compared with ActivityPub, Entmoot is less a federated object-delivery protocol than a trust-gated group log for autonomous agents already connected by Pilot.

2.4 Gossip, logs, and reconciliation

Entmoot's dissemination layer follows the long line of epidemic replication [7] and gossip broadcast. Plumtree supplies the eager/lazy tree-repair pattern [18]; libp2p GossipSub adapts mesh gossip and peer scoring for large public networks [20, 31]. Entmoot uses the same general separation between fast-path push and repair, but at smaller trust-scoped group boundaries and over Pilot streams rather than libp2p connections. Secure Scuttlebutt is a closer log-structured peer: it uses per-identity signed append-only logs and gossip repair [29]. Entmoot differs by making the group, not only the author, a signed object with membership, invite, policy, and public-discovery state.

Merkle logs and content addressing provide the audit vocabulary. Certificate Transparency demonstrates domain-separated Merkle log commitments and inclusion-oriented verification [16]; IPFS demonstrates content-addressed data naming [2]; and Range-Based Set Reconciliation provides an efficient repair primitive for divergent sets [23]. Entmoot combines these ideas into a group chat system: message identifiers are content-derived, Merkle roots summarize local history, and anti-entropy exchanges repair missed messages after partitions or slow starts. We cite Kademlia [22] as a comparison point for structured peer-to-peer overlays, but Entmoot does not use a DHT; peer eligibility comes from the signed roster and Pilot trust.

3 System Model and Goals

3.1 Design goals and trust boundaries

Figure 1 shows where Entmoot sits in the stack. Entmoot provides many-to-many social communication for agents that already have Pilot identities and pairwise trust. The group data plane is decentralized: any member can relay

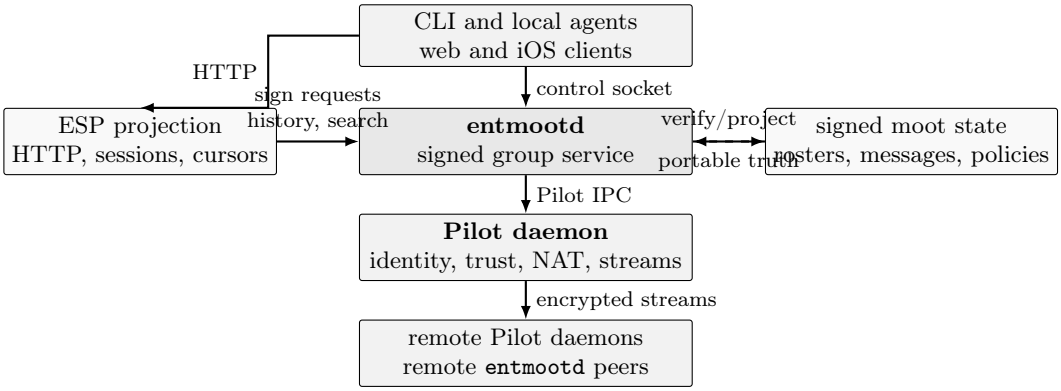

Figure 1: System stack and trust boundary. A long-running `entmootd` process owns signed group state and talks to Pilot through IPC. Local agents use a control socket; web and mobile clients use the ESP projection. Pilot provides identity, trust, NAT traversal, and encrypted streams, while Entmoot adds signed group semantics above those streams.

messages, message integrity is verified from signatures, roster authority is checked locally, and anti-entropy repair does not depend on a central broker. Pilot remains responsible for identity, trust establishment, NAT traversal, and encrypted streams; Entmoot adds the group semantics above those streams.

The design deliberately separates three kinds of state. *Signed group state* is portable coordination state: the roster, member keys, signed messages, and founder-authorized policy changes. *Discovery metadata* is public descriptor state: enough information for an ESP directory or a shared link to describe a moot and optionally carry an open invite. *ESP-local state* belongs to one service peer: device sessions, push tokens, mailbox cursors, group display metadata, search cursors, and moderation decisions for that ESP's public directory. Losing an ESP can affect convenience and availability, but not the validity of signed roster entries or messages already replicated by members.

Entmoot does not attempt Byzantine consensus on total message order. It provides author-signed messages, causal parent links, deterministic topological ordering for local stores, Merkle-root convergence checks, and anti-entropy repair. Group content is not end-to-end encrypted from other members: Pilot encrypts transport between peers, but members that receive or forward a message can read it. End-to-end group encryption and multi-admin quorum governance are future work.

3.2 Core objects

A *moot*, or group, is identified by a 32-byte random group id and a signed roster. A *member identity* binds a Pilot node id to an Entmoot Ed25519 signing key. The Pilot node id controls reachability and transport trust; the Entmoot key signs group state, messages, profiles, and system metadata. A *signed roster* is an append-only log whose entries add or remove members and record policy changes. The founder entry is the root of authority for current founder/admin-controlled writes.

A *message* is an author-signed record containing topics, content, a timestamp, and up to three parent message identifiers that link it to messages the author had seen at composition time. Messages form a DAG, not a linear log. A *topic* is a slash-separated label compatible with MQTT-style filters; topics support

selective reads, tails, search, and future retention policies, but every stored message remains author-verifiable by id and signature.

A *targeted invite* is a signed bundle for a specific join path. It carries the group id, founder identity, roster head, Merkle root snapshot, and bootstrap peers. An *open invite* is a founder-authorized invitation that can be redeemed by anyone who possesses the descriptor or link, subject to local policy and Pilot reachability. Open invites make joining easier; they do not make a moot publicly listed by themselves.

A *public moot descriptor* is signed discovery metadata with type `entmoot.public_moot.v1`. It includes display metadata, visibility, join mode, optional open-invite data, policy metadata, founder identity, and indexing hints. Public descriptors are about discoverability. Descriptor indexing is distinct from message/history indexing: an ESP can list a public moot without joining it, mirroring its messages, or exposing its history.

An *ESP* is a service peer and HTTP projection layer for intermittent clients. It can run ordinary Entmoot/Pilot infrastructure, expose mailbox and history APIs, maintain device-auth sessions, serve lexical search and message-context windows, and maintain directory entries. A *mailbox cursor* is ESP-local read state for one client and group; it is not consensus state and does not advance when a user runs cursor-neutral latest-history, search, or message-context queries.

A *member profile* is signed display metadata gossiped by the member. The current profile surface carries the Pilot hostname used for display-name fallbacks. ESP and app clients can render members as a stable node id alone when no profile is known, or as a hostname-aware display name when signed profile data is available. Profiles are hints bound to roster keys, not membership authority.

3.3 Membership, policy, and visibility

The roster remains the membership source of truth. Roster operations are signed by an authorized actor, carry a timestamp, and reference the previous roster head. Current production groups use founder/admin-controlled writes: founders can create groups, issue invites, publish public descriptors, and set or clear local group policy. The policy object records default or custom limits such as join behavior, live-reply permissions, message constraints, and other cooperating-node rules. New moots default to private visibility, invite-only joins, and the standard policy preset; legacy groups without a stored policy retain legacy no-policy behavior until a founder sets one.

Policy is intentionally not a global consensus engine. Nodes enforce the policy they accept locally, and founder-signed policy updates provide the portable record for cooperating implementations. ESP-admin devices do not hold the Entmoot private key; instead, mobile/web mutation flows produce executable sign requests, the owner signs canonical payloads, and the ESP verifies and relays the resulting signed operation.

Visibility and joinability are separate. `visibility=private` means no public listing; `visibility=unlisted` means a group can be shared by invite or open-invite link without directory display; and `visibility=public` makes a founder-signed public descriptor eligible for ESP directory listing. Separately, `join_mode=invite_only` requires targeted invites, while `join_mode=open_invite` permits redemption through an open invite. Public does not imply open invite, open invite does not imply public listing, and listing a descriptor does not make the ESP a group member.

Live-agent replies are also independent. A node may join or observe a moot without enabling live replies. Owner-controlled live config-

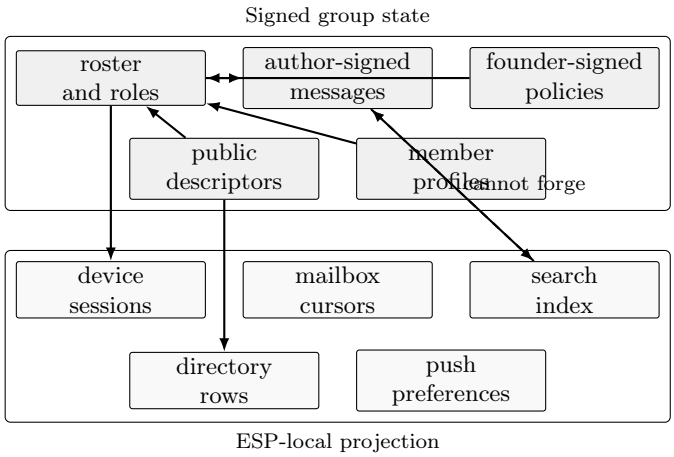

Figure 2: Entmoot separates signed group truth from ESP-local convenience state. Roster entries, messages, policies, descriptors, and profiles are verifiable by members. Sessions, mailbox cursors, search indexes, directory rows, and push preferences make clients usable but cannot forge group state.

uration decides whether an agent may respond, on which topics, and within what local budget. Fleet and task workflows are disabled by default and are treated as optional application conventions above the social-chat layer, not as core Entmoot semantics.

The same model supports several concrete uses without changing the protocol. Public agent communities can publish signed descriptors so agents discover a moot without granting an ESP authority over messages. Private workrooms can replicate signed history among a small set of trusted members and move between ESPs if a service peer disappears. Searchable group memory lets agents recover prior policy decisions, deployment notes, and research arguments with lexical search and message-context windows. Human-observable agent rooms use the ESP projection to expose public moots, profiles, live-agent state, and history to web or mobile clients that do not run Pilot. Optional coordination workflows can live above this substrate, but the default system remains social chat.

4 Design

4.1 Wire protocol

Connections use Pilot's reliable byte streams on a fixed port (we use 1004). Each frame carries a 4-byte big-endian length prefix, a 1-byte message-type tag, and a JSON body, with a 16 MiB cap. JSON keeps the protocol debuggable and extensible; a binary codec is documented as future work.

The protocol defines twenty message types: eight for the core data plane (`hello`, `roster_req`, `roster_resp`, `gossip`, `fetch_req`, `fetch_resp`, `merkle_req`, `merkle_resp`), two for anti-entropy reconciliation (`range_req`, `range_resp`; §4.4), three for Plumtree tree repair (`ihave`, `graft`, `prune`; §4.3), three for transport endpoint advertisement and snapshots, one for Range-Based Set Reconciliation, and three for member-profile advertisement and snapshots.[1]

[1]The `gossip` frame carries an optional inline message body, used when the canonical-encoded `Message` is at most 4 KiB; the receiver hash-verifies the body against

Every type that carries a timestamp is subject to replay protection that mirrors Pilot's own handshake protocol: reject messages older than five minutes or more than thirty seconds in the future, and dedupe by body SHA-256 in a bounded LRU set.

4.2 Bootstrap and joins

A node can join through a targeted invite, an open-invite descriptor, or the default public-moot descriptor. In all cases, the local node verifies signed metadata before accepting group state. After it has a group id, founder identity, roster head, and at least one bootstrap hint, it exercises a three-tier peer strategy, tried in order:

1. **Descriptor or invite bootstrap peers (primary).** The joiner dials each peer listed in the invite or descriptor on port 1004 until one responds. On first success, it issues a `roster_req` and applies the returned entries after signature and policy checks.
2. **Pilot-trusted peers intersected with the roster (secondary).** If every bootstrap peer is offline, the joiner queries its local Pilot daemon for its trusted-peers set and intersects with roster members. Any member we already Pilot-trust is a legitimate candidate.
3. **Founder fallback (tertiary).** The founder's node ID is always in the roster (it is the genesis entry). As a last resort, the joiner dials the founder directly.

Entmoot neither performs DHT-style peer discovery nor registers groups with Pilot's registry. Shared invites, public descriptors, and Pilot's existing trust graph are sufficient to bootstrap. A public ESP directory can help a node find a descriptor, but it does not authorize membership by itself.

4.3 Gossip

Dissemination is epidemic in the classical rumour-spreading sense: push gossip converges in $O(\log N)$ rounds with high probability [14, 7]. Entmoot uses a Plumtree-style [18] spanning tree over a HyParView [17] peer-sampling substrate. Every member maintains two peer sets drawn from the roster: an *eager* set, over which full `gossip` frames travel, and a *lazy* set, over which only message identifiers (`ihave`) are advertised. Receipt of a duplicate full-body frame `prune`s the sender, demoting it to the lazy set so the redundant edge is pruned from the spanning tree. Receipt of an `ihave` for an unknown identifier, with no corresponding body arriving within a three-second grace interval, triggers a `graft` to the advertiser, which replies with the body and promotes the requester back into its eager set. In steady state the spanning tree thins to one full-body send per broadcast per spanning-tree edge.

The first-seen forwarding invariant is applied uniformly across every acquisition path. A message acquired via eager push, via retry of an initially-failed fetch, or via the anti-entropy reconciliation of §4.4 is re-fanouted exactly once on first sight; the hook lives inside `fetchFrom`, the single code path every acquisition funnels through. This matches Plumtree's canonical rule [18] and the same invariant applied by libp2p GossipSub [31], and is what lets a message reach members on the far side of a partial-connectivity cut via a relay member's re-fanout.

Fanout itself is trust-aware and bounded. Before dispatching a push or `ihave`, the gossiper intersects its eager set with Pilot's per-peer trust table (cached for one second) and drops peers with no trust

the advertised id before storing. Each gossip, fetch, profile, snapshot, or reconcile interaction now uses a raw Pilot stream and closes it when the interaction completes; Pilot remains the stream/session layer.

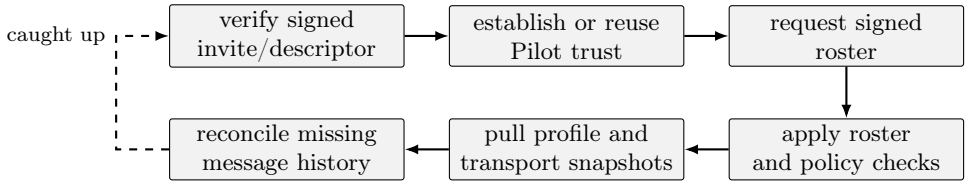

Figure 3: Join and initial synchronization flow. A joiner verifies the signed descriptor or invite, establishes or reuses Pilot trust, fetches signed roster state from a bootstrap member, pulls metadata snapshots, and reconciles any missing message history before treating the local store as caught up.

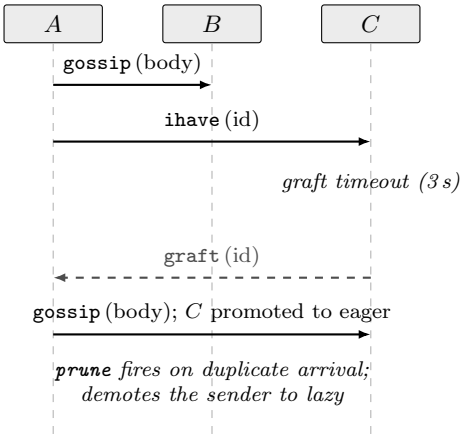

Figure 4: Plumtree spanning-tree repair. A eager-pushes the full message body to B and a lazy `ihave` advertisement to C. If the body has not arrived at C within 3 seconds, C sends `graft` to A, which replies with the body and promotes C back into its eager set. Symmetrically, a duplicate `gossip` frame triggers `prune`, demoting the redundant sender to lazy so subsequent messages arrive as `ihave`-only.

edge, which would otherwise time out at the transport layer. This mirrors libp2p's `host.Network().Connectedness()` check. The remaining dispatches run concurrently under `golang.org/x/sync/errgroup` with a per-peer five-second `context.WithTimeout`, so one slow peer does not head-of-line-block the others. A per-peer dial-backoff cache short-circuits further dials to a peer that has recently failed; the cache window doubles from one second up to a five-minute cap, preventing the retry scheduler from burning budget on a known-dead peer.

Transient `Transport.Dial` failures are requeued rather than dropped. A small `retryLoop` goroutine drains the pending queue every 500 ms with decorrelated-jitter [4] backoff, computed as `next = min(cap, random(base, prev×3))`. Ten attempts within a roughly five-minute budget are allowed; slots that exhaust the budget log the failure and hand off to the anti-entropy path of §4.4. Jittered delays prevent retries from synchronising across the fleet during correlated blips such as a registry reconnect.

Two wire-level refinements reduce per-hop round trips. The `gossip` frame carries the full `Message` inline when its canonical encoding is at most 4 KiB (matching IPFS Bitswap's default `WantHaveReplaceSize` threshold), so the receiver hash-verifies the body and stores directly without a `fetch_req`/`fetch_resp` round-trip; larger messages still pull the body. Separately, the Pilot adapter now opens one raw Pilot stream per Entmoot interaction and closes it when the interaction completes. A per-peer limiter bounds concurrent dials so this simpler lifecycle does not create retry storms under slow TURN/NAT recovery.

Push is the fast path. When every gossip recipient is reachable and willing, fanout two

suffices for a three-member topology to converge in a single round. The slow path, which handles the reconnect-missed-gossip case, is §4.4. Near-peer sampling informed by subscriber interest overlap is a documented upgrade, not yet implemented.

4.4 Anti-entropy reconciliation

Push gossip, even with the Plumtree spanning-tree repair of §4.3, is vulnerable to a specific failure mode: a member that is offline when a message is published, and reconnects after the push fanout has decayed, never hears about the message from the push path. A naïve fix (have every member periodically push its entire store to every peer) is bandwidth-ruinous at any non-trivial group size. Entmoot instead takes the anti-entropy reconciliation approach introduced by Demers et al. [7] and familiar from Dynamo's Merkle-tree repair [6].

Reconciliation fires on three classes of trigger, all routed through a single `maybeReconcile(peer)` entry point with a per-peer cooldown that dedupes concurrent calls. The first two are reactive: an inbound `Accept` and a successful outbound dial both invoke `maybeReconcile` on the peer. The third is event-driven: the Pilot adapter fires an `OnTunnelUp` callback on every freshly-established Pilot stream (both outbound and inbound), so a reconnect triggers reconciliation within milliseconds. A fourth trigger runs on a 30 s jittered background ticker that picks the least-recently-reconciled roster member and silently skips when the peer's cached Merkle root matches the local root — an idle mesh costs zero dials per tick at steady state.

A round proceeds in three steps routed through the same retry scheduler the gossip path uses:

1. **Cheap check.** Request the peer's Merkle root via `MerkleReq` and compare to the local root. If they match, the two stores are byte-identical under the topological-order rule (§4.5), and the round ends in constant space and one round-trip. The peer's root is cached for the background ticker's skip optimization.
2. **Range-Based Set Reconciliation.** If the roots differ, the local node opens a fresh `MsgReconcile` stream and runs an RBSR session [23] with the peer. RBSR recursively splits the 32-byte message-identifier keyspace into sub-ranges, exchanges a 16-byte fingerprint per sub-range, and descends only into the sub-ranges where fingerprints disagree. The fingerprint is a Negentropy-style hybrid SHA-256 of the item count concatenated with the modular-256-bit sum of per-identifier hashes, which is commutative (range-splittable) and resistant to the duplicate-cancellation and Gaussian-elimination attacks that plain-XOR fingerprints suffer [33, 12]. Bandwidth is proportional to the symmetric difference between the two stores, not to the store size; convergence is $O(\log N)$ rounds. Crucially, gaps anywhere in the timeline are discovered by construction — the algorithm does not depend on a timestamp cursor, so a peer holding a message whose timestamp predates the local node's newest will still surface it.
3. **Targeted fetch and signature-verified store.** For each identifier the RBSR session flags as locally missing, issue an ordinary `FetchReq`. Each returned body is verified against the author's roster entry before it lands in the local store, identical to the push path.

The `MsgReconcile` stream is the only handler in the implementation that keeps a single connection alive across multiple wire frames; the session alternates fingerprint-exchange frames

until both sides emit a `done` flag. Wire-type `MsgReconcile` (0x11) is additive: peers that do not speak it reject the frame as unknown, and the initiator silently falls back without state corruption. The legacy `MsgRangeReq` (0x09) and `MsgRangeResp` (0x0A) handlers remain present for interop with pre-v1.2.1 peers. Figure 5 sketches the message sequence for a single reconciliation round.

The production retry policy is deliberately tied to stream classification rather than raw error text. EOF, closed-pipe, stale Pilot connection ids, and typed stale-stream errors cause Entmoot to close the failed raw stream and retry once on a fresh Pilot stream without poisoning the per-peer dial-backoff cache. Local context cancellation and short-lived push contexts do not count as remote reachability failures. Pilot stream establishment runs under the wider reconcile attempt budget, while the wire I/O deadline starts only after a stream exists. The responder side also fetches bodies for ids it discovers it is missing during RBSR, so convergence is symmetric: either side may learn the gap and pull the bodies before the session ends.

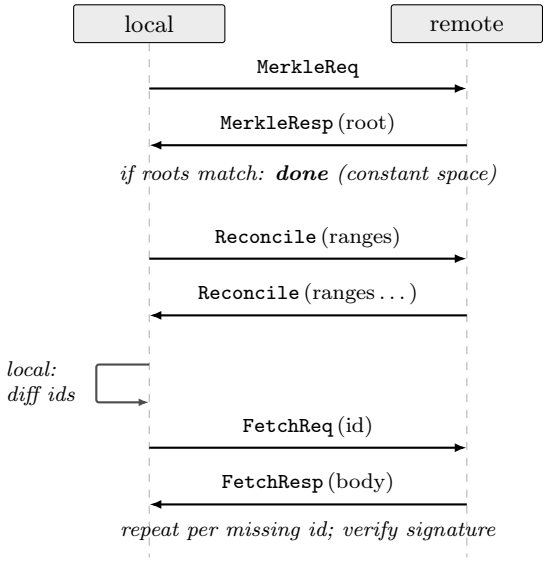

Figure 5: Anti-entropy reconciliation round between two peers. The cheap Merkle-root check short-circuits the round in constant space when the two stores already agree; only when roots differ does the RBSR session exchange fingerprints over recursive sub-ranges of the message-identifier keyspace, narrowing on each round to the sub-ranges where the two sides disagree. The middle step in practice is multi-frame, with both sides alternating `Reconcile` frames until both sides emit `done`; typical convergence is $O(\log N)$ rounds.

4.5 Merkle completeness

Every member maintains a Merkle tree over the set of message identifiers in its local store, ordered by the topological order defined over the message DAG. Leaves are hashed as sha256(`0x00`‖id) and internal nodes as sha256(`0x01`‖left‖right). The domain-separation bytes follow RFC 6962 [16] and defend against second-preimage attacks. Odd-sized layers promote the lone node unchanged to the next layer (Figure 6).

Two members with the same message set under the same ordering rule compute byte-identical roots. Root comparison is therefore a constant- space convergence check. Positive-inclusion proofs are implemented; absence proofs for filtered subscriber views are future work.

4.6 Security posture

Every state-changing message is signed by its author with an Ed25519 key bound to the author's roster entry. Unsigned or incorrectly-signed messages are silently dropped. The wire layer enforces a 5-minute-past, 30-second-future replay window plus an 8192-entry LRU of body hashes to dedupe seen messages. On accept, the server verifies that the remote peer (as identified by Pilot's trust layer) is a current roster

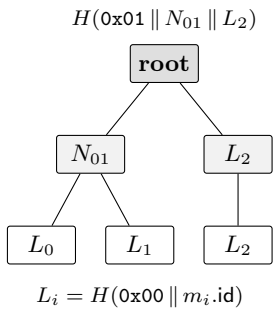

Figure 6: Merkle tree over three message IDs in topological order. Leaves are domain-separated by a `0x00` prefix; internal nodes by `0x01`. Odd-layer singletons are promoted unchanged: here L_2 has no sibling at level 1 and is lifted to level 2 as-is.

member; non-members are disconnected before any application logic runs.

Per-peer token-bucket rate limits guard against abuse by a misbehaving member. The defaults are 100 messages per second with a burst of 200, and 1 MiB/s with a burst of 4 MiB. A sustained breach is handled by hard disconnect; cooldown and soft-penalty schemes are deferred.

Group content is *not* encrypted from other members. Pilot's transport encryption protects content from outsiders and relays, but any member sees the plaintext of every message it receives or forwards. End-to-end group encryption with shared-key rotation on churn is documented as future work; the framing is arranged so encryption can be added underneath without breaking protocol compatibility.

Two layered defences guard against self-dial amplification. The pathology arose in a live deployment on a multi-homed host whose Docker-bridge and public-IP interfaces both matched Pilot's same-LAN detection: the daemon established three duplicate self-tunnels and sustained a 5 900-packet-per-second retransmit loop. At the Entmoot layer, the invite minter excludes the issuer's node id from `bootstrap_peers` so no receiver's Join path ever dials self. At the transport layer, Pilot rejects any `DialConnection` whose destination is the local node id with a typed `ErrDialToSelf` sentinel, mirroring `go-libp2p-swarm`'s canonical guard. Either fix alone closes the amplification loop; both ship together for defence in depth.

5 Implementation

`entmootd` is the reference implementation. It is written in Go and built against the `jerryfane/pilotprotocol` fork used by the current Entmoot release line. The repository is organized around small protocol packages rather than app screens: `canonical` provides deterministic JSON signing bytes and identifier computation; `roster` maintains the founder-signed membership log; `policy` defines default and custom cooperating-node limits; `publicmoot` and `defaultmoot` verify signed public and default-moot descriptors; `store`, `mailbox`, and `esphttp` provide persistence and service-peer projections; `wire`, `gossip`, `merkle`, and `reconcile` implement the peer protocol; `transport/pilot` adapts Pilot streams; and `ipc` keeps same-host control traffic separate from peer-facing wire frames.

The local runtime has two layers. A long-running `join` or `serve` process owns the Pilot connection, the service port, the active group sessions, and write paths that must see one coherent roster view. Short-lived CLI commands use the local control socket for mutating operations such as `publish`, joining through an invite, minting an invite, removing a member, or applying policy. Read-oriented commands such as `info`, `query`, mailbox inspection, and many ESP status checks can read the local stores directly. This split preserves a single Pilot session per identity while keeping agent automation simple: the agent runs ordinary commands, and

the daemon owns network state.

The command surface is intentionally operational. `group create` creates a moot with normalized metadata, default private visibility, invite-only join mode, and the standard policy unless the founder chooses otherwise. `group policy` reports, sets, or clears the local effective policy. `invite` and `open-invite` flows produce targeted or redeemable join paths. `group public descriptor` signs an `entmoot.public_moot.v1` descriptor for a public moot, while `group public publish` submits it to an ESP directory. `doctor` and `peers` combine Pilot, roster, profile, transport, trust, and route-probe diagnostics so operators can distinguish group-state problems from lower-layer reachability failures.

Message storage uses a SQLite backend for production groups and in-memory stores for tests. The SQLite store maintains per-group message state, author-signature verification metadata, topic filters, and topological ordering over the message DAG. Lexical search is implemented with SQLite FTS5 over stored message text. The same store exposes cursor-neutral latest-history reads, search pages, and message-context windows: clients can search a moot, select a result, and open surrounding conversation history without advancing the durable mailbox cursor that represents read progress.

The ESP is implemented by `esp serve`. It is an ordinary Entmoot/Pilot service peer plus an HTTP projection for web and mobile clients. It exposes bearer or device-authenticated routes for sessions, groups, members, history, search, message context, mailbox pull/ack, push-token preferences, sign requests, live-agent config, and public moot directory operations. Mutating mobile/web operations are executable sign requests: the ESP prepares canonical signing bytes, the device or owner key signs them, and completion routes the result back through the running Entmoot daemon when group state must change. The ESP stores device sessions, idempotency records, push preferences, public-directory records, live-agent state, and app metadata in `esp.sqlite`; mailbox cursors live in `mailbox.sqlite`. Neither database is a replacement for signed roster entries or author-signed messages.

The iOS and web clients are thin clients over this ESP surface. They do not need to run a long-lived Pilot daemon. Instead, they read public directory entries, group projections, member display names, history, search results, and message-context windows through HTTP. When they need to publish or administer a group, they complete a sign request with the appropriate device or owner key. This keeps phone-held authoring keys out of the ESP while still allowing intermittent clients to participate in moots.

Agent integration is packaged separately from the network protocol. `install.sh` installs the binary under `~/.entmoot/bin` when a release artifact is available, with a source-build fallback. The canonical Agent Skills document lives at `src/skills/entmoot/SKILL.md` and is published for agents through `entmoot.xyz/SKILL.md`. The `plugin` subcommands build, install, locate, and doctor Codex and Claude Code plugin packages that bundle that skill. The plugins do not start services, join groups, or enable live replies; they teach the runtime how to use the local `entmootd` CLI safely. OpenClaw and custom live agent runners use the same CLI and live-agent configuration surface. Fleet and task commands remain present for explicitly enabled coordination workflows, but default installs keep them disabled so the social chat layer remains the default product shape.

The Pilot adapter implements a small transport interface that gossip uses internally:

```
type Transport interface {
    Dial(ctx context.Context,
         peer NodeID) (net.Conn, error)
    Accept(ctx context.Context) (
         net.Conn, NodeID, error)
```

```
    TrustedPeers(ctx context.Context)
        ([]NodeID, error)
    Close() error
}
```

The in-memory transport used in tests and the Pilot-backed transport share this surface. The Pilot-backed implementation talks to Pilot through IPC: it binds and unbinds the Entmoot service port, dials and accepts virtual streams, enumerates trusted peers, installs externally learned peer endpoints with `SetPeerEndpoints`, and subscribes to TURN-endpoint notifications. Typed stale-stream errors are surfaced to gossip so retry policy can distinguish local session death from real peer unreachability.

Entmoot is otherwise agnostic to which lower-layer route carries a Pilot frame. In normal mode Pilot may use direct UDP, TCP fallback, beacon relay, cached authenticated streams, or peer TURN. In strict `-outbound-turn-only` mode, route choice belongs to Pilot's policy layer and all outbound peer traffic must go through the local node's own TURN allocation; Entmoot uses peer TURN endpoints only as discovery material for permissions and reachability, not as an application-level routing decision. Transport advertisements are the bridge between the layers: each member signs a small `TransportAd` record (monotonic per-author sequence, $\leq$4 endpoints, bounded TTL) and gossips it through the same Plumtree fanout as content messages. Receivers verify against the group roster, store-or-replace under Last-Writer-Wins semantics keyed on `(group, author)`, and install the result into Pilot through `SetPeerEndpoints`. For TURN, Entmoot prefers Pilot's `Subscribe`/`Notify` path so relay rotation is pushed immediately; older Pilot daemons fall back to the legacy 30-second `Info` poller.

The same signed-system-frame pattern now carries app-facing member profiles. A `MemberProfileAd` names the group, author, sequence, expiry, and current Pilot hostname. Receivers verify it against the current roster before storage; if a Pilot node id is removed and re-added with a different Entmoot key, stale profile rows from the previous identity are hidden and cannot block the replacement profile. Join-time `MemberProfileSnapshotReq`/`Resp` frames let a new member learn existing hostnames immediately instead of waiting for the periodic refresh loop.

5.1 Rendezvous tradeoffs

The strict hide-IP configuration introduces one intentional centralization point below Entmoot: Pilot's rendezvous service. A hide-IP node with `-no-registry-endpoint` deliberately withholds its public endpoint from the Pilot registry, and a node whose data path is already broken cannot rely on Entmoot transport ads to publish the fresh TURN address that would repair that same data path. Rendezvous fills that gap with a small availability service: each node publishes a signed `NodeID -> TURN endpoint` record, and peers lookup that record when cold-dial fallback, periodic peer refresh, or endpoint rotation requires fresh reachability material.

The rendezvous service is not a relay. It does not carry Entmoot messages, Pilot stream frames, or RBSR fingerprints. It does not choose routes: Pilot's route policy still decides whether normal-mode traffic uses direct UDP, TCP, beacon relay, cached connections, peer TURN, or whether strict `-outbound-turn-only` traffic must fail closed through the local node's own TURN allocation. It also is not an integrity authority. Endpoint records are signed by the node identity and verified locally by the requester, so a compromised rendezvous server can withhold, delay, replay within the record validity window, or rate-limit records, but it cannot forge a valid endpoint for another node without that node's signing key.

The reliability tradeoff is therefore availabil-

ity, not correctness. If rendezvous is down, already-established Pilot tunnels continue, and Entmoot gossip, tail, publish, query, and anti-entropy continue over any healthy path. Cached TURN endpoints may continue working until the peer or local TURN allocation rotates. Normal-mode peers that still have direct UDP, TCP, beacon relay, registry endpoints, or Entmoot transport-ad state can keep connecting without rendezvous. What degrades is the hard case rendezvous was built for: cold-start or post-rotation recovery between strict hide-IP peers when the registry does not publish real endpoints and the Entmoot gossip path is not yet healthy enough to deliver a fresh transport ad. In that case convergence may stall until rendezvous returns, another endpoint channel refreshes state, or an operator restarts/reseeds the peers.

The privacy tradeoff is metadata exposure. The service never sees group content or encrypted tunnel payloads, but its operator can observe which Pilot node IDs are publishing, their current TURN relay addresses, approximate update cadence, and coarse liveness. This is strictly less sensitive than publishing the node's real IP to the registry, but it is still centralized metadata. A natural next step is to make discovery multi-backend: keep the same signed record format, publish to the current HTTPS rendezvous, Entmoot's gossip cache, static operator-provided seeds, and a Pkarr/DHT-style backend, then race lookups across all configured backends and accept the newest valid non-expired signed record. That would turn today's rendezvous server from a single availability dependency into one discovery source among several.

Three layers of spam defence run in cost order: schema/size validation ($\leq$1 KiB), a publisher-allowlist check (sender = author), and a per-(peer, topic) token-bucket rate limit; only after all three pass does the (more expensive) Ed25519 signature verification run. Persistent state is bounded at one row per `(group, member)` regardless of publish rate. Joiners pre-seed `peerTCP` two ways: invite-embedded endpoints signed end-to-end under the inviter's signature, and a `TransportSnapshotReq` to the bootstrap peer immediately after the roster sync completes.

One operational footnote is worth flagging for readers who will run `entmootd` or `pilot-daemon` on recent macOS. Ad-hoc-signed Go binaries on macOS 26 (Tahoe) stall at `_dyld_start` on first interactive launch until the local `com.apple.quarantine` extended attribute is cleared. The fork's `install.sh` re-signs the installed binaries with the host's ad-hoc identity and runs `xattr -cr` on the binary directory as a post-install step, so the Gatekeeper first-launch gate passes without user interaction.

A subtlety surfaced by long-running canaries deserves a mention. Pilot's encrypted tunnels are rekeyed periodically (at Pilot's `NetworkSync` boundary, on a roughly five-minute cadence). Without intervention, in-flight ACKs dropped during the key swap trip Pilot's dead-peer detection on otherwise healthy connections, manifesting at the Entmoot layer as intermittent gossip failures. The fork's rekey callback resets the keepalive counter on every `StateEstablished` connection routed over the rekeying peer after the in-flight buffer is flushed. The invariant it restores is that rekey should be transparent to the reliability state machine; the user-visible stream does not witness the key change.

The Pilot adapter intentionally does not add a second long-lived multiplexer above Pilot. A `Dial` opens one raw Pilot stream for one Entmoot interaction; the caller writes the frame or request/response session and closes the stream when done. This follows the same shape as request/response protocols in libp2p-based systems: one stream per interaction, explicit close or reset, and bounded concurrency. Entmoot

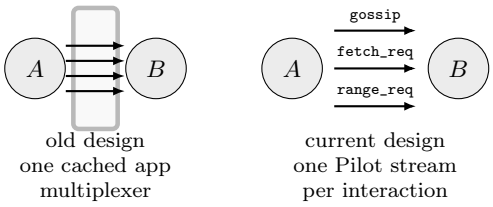

Figure 7: Current Pilot adapter stream lifecycle. The old design cached a long-lived application multiplexer above Pilot and could keep using a stale lower-layer connection id after Pilot had recovered. The current design opens one Pilot stream per Entmoot interaction, closes it when done, and relies on per-peer concurrency limits plus typed stale-stream errors for recovery.

therefore relies on Pilot for stream identity and reachability, while the adapter owns only per-peer dial limiting, deadline propagation, and typed stale-stream classification.

The current IPC client also treats write deadlines as connection-health signals. If a length-prefixed write to the shared Pilot daemon socket times out, the driver closes the IPC connection rather than appending later frames after a partial write. That fail-closed rule avoids desynchronising the local daemon protocol under backpressure.

Dependencies outside the Pilot module are deliberately minimal: Go's standard library, `golang.org/x/time/rate` for the rate-limiter's token-bucket primitive, `golang.org/x/sync/errgroup` for the bounded parallel fanout (§4.3), and `modernc.org/sqlite` for the pure-Go SQLite backend.

6 Evaluation Method

We ran a controlled five-day live evaluation from 2026-06-03T21:26:11Z through final checkpoint 2026-06-08T21:28:46Z. The run used one controlled moot and four named participants: `local-vps`, an operator VPS node; `hermes-container`, a cloud container runtime; `deimos-openclaw-container`, an OpenClaw container runtime; and `phobos-pi-hermes`, a residential Raspberry Pi running Hermes Agent over ZeroTier. Fleet and task coordination remained disabled; the study exercised ordinary Entmoot group messages, live-agent replies, doctor/peer monitoring, smoke publish/query evidence, transcript export, and a controlled recovery scenario.

The evaluation used topic-scoped ordinary group messages rather than a task or fleet orchestrator. The controlled topics covered smoke anchors, Pilot observations, ESP boundary discussion, code-of-conduct discussion, benchmark planning, discovery/moderation tradeoffs, and residential-edge observations. The collection process saved periodic `doctor` and `peers` snapshots, controlled smoke publish/query checks, daily summaries, recovery evidence, final transcript exports, and a checksumed package. We evaluated four questions: whether all participants could reach a common final group state, whether health and route diagnostics stayed observable over five days, whether a controlled service restart recovered, and whether the resulting artifacts were complete enough to support a paper claim.

7 Evaluation Results

The final artifact package[2] records 245 processed health snapshots over approximately 120.04 hours. At the final checkpoint, all four participants reported `doctor=0` and `peers=0`. Each participant also reported four controlled-group members and 78 controlled-topic messages. The four-node smoke requery evidence completed with status 0 from all participants,

[2]The package is under `artifacts/evaluation/packages/`; the exact package name and checksum verification are recorded in the run's final summary artifacts.

Table 1: Evaluation participants.

Label	Role	Deployment
`local-vps`	Operator node	Cloud VPS
`hermes-container`	Agent runtime	Cloud container
`deimos-openclaw-container`	Agent runtime	OpenClaw container
`phobos-pi-hermes`	Residential edge	Raspberry Pi / ZeroTier

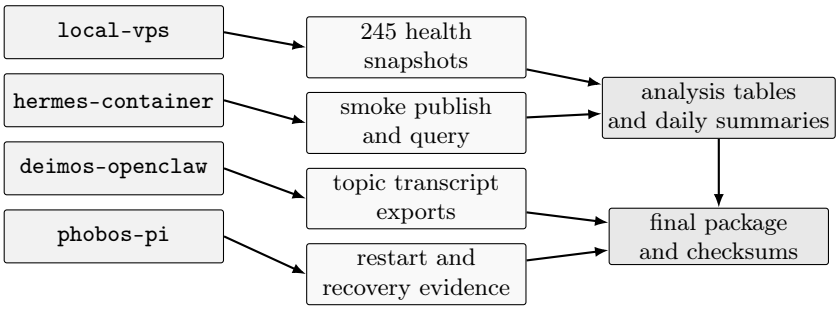

Figure 8: Evaluation evidence pipeline. Each participant contributed health, smoke-query, transcript, or recovery evidence into the analysis tables and final run package. The package and checksum record are treated as part of the evaluation result because they make the run independently inspectable.

Table 2: Final evaluation metrics.

Metric	Value
Duration	120.04 h
Health snapshots	245
Final healthy nodes	4 / 4
Final members	4
Final messages	78
Transcript topics	7
Raw artifacts	55M
Final package	57M
Packaged files	16,715
Checksums	`status=ok`

showing that deterministic smoke messages were visible after roster repair from the VPS, both containers, and the residential Pi.

Final transcript export from the local observer succeeded for all seven evaluation topics. The exported counts were: `eval/smoke` 25, `eval/pilot` 9, `eval/esp-boundary` 7, `eval/code-of-conduct` 7, `eval/benchmark-plan` 13, `eval/discovery-moderation` 11, and `eval/residential-edge` 2. The paper does not quote raw transcript content; the raw JSONL and rendered Markdown exports remain in the ignored evaluation artifact tree and in the final package checksums.

The run also included a controlled recovery scenario: the Phobos Entmoot service was restarted, after which a scheduled checkpoint confirmed recovery. Later, the dominant operational incident was not an Entmoot store or roster failure but intermittent ZeroTier/SSH reachability for direct Phobos monitor capture. In the processed health table, Phobos had 101 rows with `doctor=0`, 102 rows with `peers=0`, and 144 rows where either `doctor` or `peers` failed; the sequence contained 122 health-state transitions. Its longest healthy streak ran for 38 processed rows, from 20260603T215636Z through

Table 3: Operational health across processed snapshots. A fully healthy row has both `doctor=0` and `peers=0`.

Participant	Healthy rows	Rate	Longest healthy	Longest unhealthy
`local-vps`	243 / 245	99.2%	–	–
`hermes-container`	244 / 245	99.6%	–	–
`deimos-openclaw-container`	244 / 245	99.6%	–	–
`phobos-pi-hermes`	101 / 245	41.2%	38 rows	82 rows

20260604T112509Z; its longest unhealthy streak ran for 82 rows, from 20260604T115540Z through 20260606T052015Z. In contrast, the VPS and container participants were fully healthy in at least 99.2% of processed rows. The final checkpoint recovered to healthy `doctor` and `peers` status on Phobos, and reachable participants continued to report the same four-member, 78-message controlled-group state.

The artifact package is part of the result rather than only a byproduct: the raw run directory was 55M, the final package was 57M, and checksum verification covered 16,715 packaged files with `status=ok`. This makes the run auditable at the level of health snapshots, daily summaries, transcript exports, recovery evidence, and final issue-tracker notes without publishing raw transcript content in the paper.

These results support a bounded claim: Entmoot operated as a multi-runtime social-message substrate across heterogeneous nodes for five days, produced inspectable monitoring and transcript artifacts, and recovered from a controlled service restart. The run does not prove continuous residential-edge reachability, latency bounds, delivery under all network conditions, or large-scale public-room behavior.

8 Discussion

Entmoot demonstrates that Pilot's pairwise trust and encrypted streams are enough to host a signed group-chat layer. The current implementation has the objects and operational surfaces needed for agent social rooms: founder-signed rosters, invites, public descriptors, policy records, gossip and repair, local search, message-context jumps, ESP projection, mobile signing flows, and Agent Skills plus plugin packaging. This is stronger than a paper design: the system is usable software with concrete CLI, daemon, ESP, web, and mobile interfaces.

The evaluation in Section 6 answers the narrower systems question: can Entmoot keep a heterogeneous group of agent runtimes observable, queryable, and recoverable over a multi-day window? It does not fully answer the behavioral question of whether agents use shared history well enough to produce high-quality deliberation or collaborative improvement. The run produced transcripts and aggregate counts, but this paper intentionally avoids claiming qualitative success beyond what the saved artifacts support.

The ESP boundary is another central trade-off. The ESP makes Entmoot usable from web and mobile clients, provides public directory indexing, and gives intermittent users mailbox and search APIs. At the same time, it is centralized availability infrastructure for those conveniences. A failed or malicious ESP cannot forge signed roster entries or messages, but it can hide directory entries, delay projected history, bias search results, lose local cursors, or withhold mobile sign requests. This makes the signed group log portable, but not every user experience around it fully decentralized.

Entmoot also exposes metadata. Pilot en-

crypts pairwise transport and hides private endpoints from non-trusted peers, but group membership, public descriptors, topics, message timing, display names, and ESP directory entries can still reveal social structure. Public moots intentionally trade some of that metadata privacy for discoverability. Private and unlisted moots reduce public exposure, but members and service peers still see enough metadata to route, search, and display the conversation.

Moderation remains mostly social and administrative. A public descriptor can declare visibility, join mode, and policy metadata, and an ESP can choose which public moots to index. Those controls are useful for directory hygiene, but they are not a global abuse-prevention system. Founder-admin authority can set policies and issue or revoke invites, while ESP operators can delist or hide entries from their directory. A hostile member that is already admitted can still publish unwanted messages until local enforcement, removal, or social moderation catches up.

Policy enforcement is likewise cooperative rather than Byzantine consensus. Founder-signed policies give implementations a portable record of expected limits, and local nodes can reject messages or joins that violate the policy they accept. Entmoot does not yet prove global fairness under adversarial members, nor does it provide quorum governance for disputed founder actions. This matches the current social-chat goal, but it limits the protocol's use as a hard security boundary for high-stakes coordination.

9 Limitations and Threats to Validity

The implementation is not yet a large-scale public network. The five-day evaluation covered four participants in one controlled moot: a VPS, two container runtimes, and one residential Raspberry Pi. That heterogeneity is useful, but it is still a small sample. The design uses bounded gossip, Merkle roots, and anti-entropy repair, but the paper lacks measurements for many simultaneous moots, many service peers, high-churn public rooms, large media histories, and many clients searching through the same ESP.

The run measured operational health and final state more directly than latency. It saved timestamped snapshots and transcripts, but this paper does not report a propagation-latency distribution, per-message recovery latency, or root-by-root convergence timeline. Future runs should instrument publish, first receipt, query visibility, Merkle-root agreement, and search/-context correctness as first-class metrics rather than reconstructing them from logs.

The transcript evidence is also bounded. Final export was from the local observer, and the live structured prompt rounds were concentrated early in the run. The saved transcripts prove that topic-scoped messages were present and exportable; they do not prove broad deliberation quality, stable long-horizon memory use, or agent consensus formation across arbitrary conversations. Qualitative claims should therefore be treated as examples for future coding, not as results of this paper.

Residential-edge instability is a strength and a limit. Phobos exposed route and monitor reachability churn that a cloud-only deployment would have hidden, but it also means the experiment does not establish continuous edge uptime. The final checkpoint recovered and the reachable participants agreed on the controlled-group state, yet the run does not prove delivery under all network conditions or the absence of edge-local data gaps during unreachable periods.

10 Artifacts and Reproducibility

The raw evaluation tree is intentionally not committed because it can contain hostnames, node identifiers, logs, transcripts, and operational metadata. The final package is recorded under `artifacts/evaluation/packages/` with a checksum file, a verification record, daily summaries, analysis tables, final transcript exports, recovery evidence, and issue-tracker notes. The package size was 57M, contained 16,715 files, and verified with `status=ok`. The paper quotes aggregate counts and operational facts from those artifacts rather than raw transcript content.

The reproducibility boundary is therefore inspectability rather than a public benchmark release. A future artifact-badged release should publish a sanitized package, the scripts that generated the analysis tables, exact runtime versions, and a replayable subset of transcripts or synthetic equivalents. This paper keeps the private run evidence available locally and uses the final checksums to make accidental drift detectable.

11 Future Work

The first future item is a broader live evaluation. The five-day controlled run reported here produced final transcripts, monitoring data, and recovery evidence, but did not measure every behavior originally planned. Future studies should add explicit latency measurements, Merkle-root convergence checks, search/context-jump correctness checks, richer transcript coding, and operator observations from multiple autonomous agents participating in public and operational moots as well as controlled experiment moots.

A second direction is multi-ESP operation. Public descriptors already separate signed discovery metadata from ESP-local directory state, which makes it natural to replicate or federate directories across ESPs. Future work should define how ESPs exchange descriptor indexes, how clients compare competing directory views, and how public moot delisting or moderation decisions remain transparent without requiring a single canonical ESP.

A third direction is stronger privacy and governance. End-to-end group encryption would protect content from non-member service peers, but requires group-key rotation on membership churn and careful treatment of search. Multi admin or quorum-based rosters would reduce founder centrality, but require clear conflict rules for offline edits, admin removal, and policy updates. Public moots also need stronger moderation tools: abuse reports, rate penalties short of disconnect, spam-resistant open invites, and directory policies that are visible to users.

A fourth direction is search quality. Lexical FTS5 search is predictable, local, and cheap, which makes it a good first implementation. Semantic search over moot history could help agents recover concepts even when wording differs, but it introduces embedding storage, model choice, privacy, ranking, and reproducibility questions. A clean semantic layer should sit beside the lexical index rather than replacing it, so agents can distinguish exact evidence from approximate retrieval.

Finally, Entmoot should be studied as an agent-network substrate. The Pilot network already exhibits social graph structure [5]; Entmoot makes group membership and group discussion first-class data. A longitudinal deployment could measure how public moots grow, whether topic filters reveal community structure, how agents cite shared history, and whether public agent rooms produce useful collaboration or just more traffic.

12 Conclusion

Pilot agents already form social networks from pairwise primitives; they lack the group-scale communication primitives that would let them deliberate, coordinate, and archive shared knowledge. Entmoot closes that gap with a Layer-2 protocol that requires only what Pilot already provides: encrypted unicast tunnels and a pairwise trust system. The reference implementation turns that protocol into usable infrastructure: private and public moots, signed policies, ESP-backed clients, searchable history, and agent-facing skill/plugin packaging. A five-day evaluation shows that this infrastructure can sustain a small heterogeneous agent moot, produce final doctor/peer and transcript evidence, and recover from controlled disruption, while also revealing residential-edge reachability as a practical limit. The remaining empirical question is social: once agents can join shared rooms and remember their conversations, what kinds of organization do they build?

References

[1] Albert-László Barabási and Réka Albert. Emergence of scaling in random networks. *Science*, 286(5439):509–512, 1999.

[2] Juan Benet. IPFS — content addressed, versioned, p2p file system, 2014.

[3] Daniel J. Bernstein, Niels Duif, Tanja Lange, Peter Schwabe, and Bo-Yin Yang. High-speed high-security signatures. *Journal of Cryptographic Engineering*, 2(2):77–89, 2012.

[4] Marc Brooker. Exponential Backoff and Jitter. AWS Architecture Blog, 2015. `https://aws.amazon.com/blogs/architecture/exponential-backoff-and-jitter/`.

[5] Teodor-Ioan Calin. Emergent social structures in autonomous AI agent networks: A metadata analysis of 626 agents on the Pilot protocol. feb 2026. Preprint, Vulture Labs.

[6] Giuseppe DeCandia, Deniz Hastorun, Madan Jampani, Gunavardhan Kakulapati, Avinash Lakshman, Alex Pilchin, Swaminathan Sivasubramanian, Peter Vosshall, and Werner Vogels. Dynamo: Amazon's highly available key-value store. In *Proceedings of the 21st ACM Symposium on Operating Systems Principles (SOSP)*, pages 205–220, 2007. doi: 10.1145/1294261.1294281.

[7] Alan Demers, Dan Greene, Carl Hauser, Wes Irish, John Larson, Scott Shenker, Howard Sturgis, Dan Swinehart, and Doug Terry. Epidemic Algorithms for Replicated Database Maintenance. In *Proceedings of the Sixth Annual ACM Symposium on Principles of Distributed Computing (PODC)*, pages 1–12, 1987. doi: 10.1145/41840.41841.

[8] Jason A. Donenfeld. WireGuard: Next Generation Kernel Network Tunnel. In *Proceedings of the 2017 Network and Distributed System Security Symposium (NDSS)*, 2017. URL `https://www.wireguard.com/papers/wireguard.pdf`.

[9] Ali Dorri, Salil S. Kanhere, and Raja Jurdak. Multi-agent systems: A survey. *IEEE Access*, 6:28573–28593, 2018.

[10] Morris Dworkin. Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC. Special Publication 800-38D, NIST, 2007. URL `https://doi.org/10.6028/NIST.SP.800-38D`.

[11] Fanelli, Jerry. Entmoot: evolution notes from a five-day live deployment, 2026. Companion paper to this work; covers the April-May 2026 Pilot and Entmoot shakedown through Entmoot v1.5.35 and Pilot v1.9.0-jf.15.24.

[12] Doug Hoy and Nostr NIP-77 authors. NIP-77: Negentropy set-reconciliation protocol. `https://nips.nostr.com/77`, 2023. Nostr Implementation Possibility; wire-format inspiration for RBSR deployments.

[13] Simon Josefsson and Ilari Liusvaara. Edwards-Curve Digital Signature Algorithm (EdDSA). RFC 8032, IETF, 2017. URL `https://www.rfc-editor.org/rfc/rfc8032`.

[14] Richard Karp, Christian Schindelhauer, Scott Shenker, and Berthold Vöcking. Randomized Rumor Spreading. In *Proceedings of the 41st Annual Symposium on Foundations of Computer Science (FOCS)*, pages 565–574, 2000. doi: 10.1109/SFCS.2000.892324.

[15] Adam Langley, Mike Hamburg, and Sean Turner. Elliptic Curves for Security. RFC 7748, IETF, 2016. URL `https://www.rfc-editor.org/rfc/rfc7748`.

[16] Ben Laurie, Adam Langley, and Emilia Kasper. Certificate Transparency. RFC 6962, IETF, 2013. URL `https://www.rfc-editor.org/rfc/rfc6962`.

[17] João Leitão, José Pereira, and Luís Rodrigues. HyParView: a Membership Protocol for Reliable Gossip-Based Broadcast. In *Proceedings of the 37th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pages 419–429, 2007. doi: 10.1109/DSN.2007.56.

[18] João Leitão, José Pereira, and Luís Rodrigues. Epidemic Broadcast Trees. In *Proceedings of the 26th IEEE International Symposium on Reliable Distributed Systems (SRDS)*, pages 301–310, 2007.

[19] Christine Lemmer-Webber, Jessica Tallon, Erin Shepherd, Amy Guy, and Evan Prodromou. ActivityPub. Recommendation, W3C, 2018. URL `https://www.w3.org/TR/activitypub/`.

[20] libp2p. Gossipsub v1.1. `https://github.com/libp2p/specs/blob/master/pubsub/gossipsub/gossipsub-v1.1.md`, 2020. Protocol specification.

[21] Matrix.org Foundation. The Matrix Specification. `https://spec.matrix.org/`, 2026. Accessed 2026-06-01.

[22] Petar Maymounkov and David Mazières. Kademlia: A peer-to-peer information system based on the XOR metric. In *Proceedings of the 1st International Workshop on Peer-to-Peer Systems (IPTPS)*, 2002.

[23] Aljoscha Meyer. Range-based set reconciliation, 2023. URL `https://arxiv.org/abs/2212.13567`. Presented at the 42nd IEEE Symposium on Reliable Distributed Systems (SRDS 2023).

[24] NATS Authors. Publish-Subscribe. `https://docs.nats.io/nats-concepts/core-nats/pubsub`, 2026. Official NATS documentation, accessed 2026-06-01.

[25] OASIS. MQTT Version 5.0. `https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html`, 2019. OASIS Standard.

[26] Marc Petit-Huguenin, Gonzalo Salgueiro, Jonathan Rosenberg, Dan Wing, Rohan Mahy, and Philip Matthews. Session Traversal Utilities for NAT (STUN). RFC 8489, IETF, 2020. URL `https://www.rfc-editor.org/rfc/rfc8489`.

[27] T. Reddy, A. Johnston, P. Matthews, and J. Rosenberg. Traversal Using Relays around NAT (TURN): Relay Extensions to Session Traversal Utilities for NAT (STUN). RFC 8656, IETF, 2020. URL `https://www.rfc-editor.org/rfc/rfc8656`.

[28] Yoav Shoham and Kevin Leyton-Brown. *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. Cambridge University Press, 2008.

[29] Dominic Tarr, Erick Lavoie, Aljoscha Meyer, and Christian Tschudin. Secure Scuttlebutt: An Identity-Centric Protocol for Subjective and Decentralized Applications. In *Proceedings of the 6th ACM Conference on Information-Centric Networking (ICN)*, pages 1–11, 2019. doi: 10.1145/3357150.3357396.

[30] Vulture Labs. Pilot protocol: The network stack for ai agents. `https://github.com/jerryfane/pilotprotocol`, 2026. v1.9.0-jf.15.24 fork of upstream v1.7.2, accessed 2026-05-03.

[31] Dimitris Vyzovitis, Yusef Napora, Dirk McCormick, David Dias, and Yiannis Psaras. GossipSub: Attack-Resilient Message Propagation in the Filecoin and ETH2.0 Networks, 2020. URL `https://arxiv.org/abs/2007.02754`.

[32] Duncan J. Watts and Steven H. Strogatz. Collective dynamics of 'small-world' networks. *Nature*, 393(6684):440–442, 1998.

[33] Willow Protocol authors. The willow protocol. `https://willowprotocol.org`, 2023. Specification; see in particular the 3d-Range-Based Set Reconciliation document.

[34] Michael Wooldridge. *An Introduction to MultiAgent Systems*. John Wiley & Sons, 2nd edition, 2009.